Open Mercato on Windows

One-command development environment — Setup & Troubleshooting Manual

1. What this is

One command turns a Windows machine into a complete Open Mercato development environment. That command is the cross-platform **starter** (`@open-mercato/starter`). It runs a read-only checkup, provisions everything it fully controls (a private Node.js, dependencies, secrets, corporate certificates), brings up the supporting services in Docker, initializes the database, and launches the app.

By default the starter uses **hybrid mode**, which is the fastest way to develop:

Runs on your machine (natively, via `yarn dev`)	Runs in Docker containers
The **application** (port 3000, hot reload), the **MCP server** (port 3001), queue workers, and the scheduler	The **OpenCode AI agent** (port 4096), **PostgreSQL**, **Redis**, and **Meilisearch**

Because the app runs natively, editor IntelliSense works out of the box and iteration is instant. The database is created and seeded automatically. The AI assistant (`Ctrl` + `K` in the app) works once you provide an LLM provider API key during setup.

Every command is **idempotent**: re-running the starter is always safe, skips what is already done, and repairs what is missing.

> **Prefer everything in containers?** Run the starter with `up --mode docker` — the app and MCP server then run inside Docker too (useful on machines where running Node workloads directly on the host is not permitted). The chosen mode is remembered. This manual describes the default **hybrid** mode unless noted.

2. Requirements

The starter follows a strict rule: **it never installs system-level software.** It installs only what it fully owns (a private Node.js, Yarn via corepack, `.env` files, certificate bundles, repository state). Anything that needs administrator rights or a Windows feature is *detected and explained* — never changed silently. The read-only `doctor` command prints an exact "hand this to IT" sheet for those items.

Requirement	Who provides it	Details
Windows	—	Windows 10 2004+ (build 19041, incl. 22H2/Enterprise) or Windows 11
CPU virtualization	You / IT	Intel VT-x / AMD-V enabled in BIOS/UEFI (usually already on) — required by WSL2
WSL2	You / IT	Required by both Docker Desktop and Rancher Desktop. Enabling it needs admin + a reboot, so the starter proposes it but does not enable it.

Container runtime	You / IT	**Docker Desktop** or **Rancher Desktop** — must be installed and running. The starter detects it and, if missing, gives you the exact install command; it does not install it for you.
Git	You / IT	Needed to obtain and update the code (user-scope install works without admin).
C++ build toolchain	You / IT	**Hybrid mode only.** A few native modules (better-sqlite3, isolated-vm, …) compile on the host during install, which needs **Visual Studio 2022 Build Tools** with the "Desktop development with C++" workload. `doctor` checks for it and prints the exact install command. Not needed in `--mode docker` (native modules build inside the container).
Memory / disk	—	**16 GB RAM recommended, 12 GB minimum**; ~20 GB free disk for the first run, 40 GB+ comfortable
LLM API key	You	One of: OpenAI, Anthropic, Google, Azure OpenAI / AI Foundry, OpenRouter, DeepInfra, Groq, Together, Fireworks, LiteLLM, Ollama, LM Studio

> **You do NOT need to pre-install Node.js, Yarn, or winget.** The starter's bootstrap installs a private, checksum-verified **Node 24 without administrator rights** (a portable copy under `%LOCALAPPDATA%\OpenMercato`, added to `PATH` for that session only), then activates Yarn through corepack. Nothing pollutes your global environment. (The Visual C++ Build Tools in the table above are the one exception — hybrid mode compiles a few native modules on the host and cannot install that toolchain for you; run `--mode docker` to avoid it entirely.)

> **Not sure a locked-down machine will work?** Run `yarn om doctor` (or `npx @open-mercato/starter doctor`) first. It is read-only — it changes nothing — and audits Node, Git, the container runtime, WSL2, hardware, network egress (proxy / TLS interception), `localhost` resolution, ports, and the clone location, ending with a verdict and an IT hand-off sheet for the admin-gated items.

3. Quick start

The one-line bootstrap (installs Node for you, then runs the starter)

These commands work on a machine with **nothing installed but the OS** (and a container runtime — see section 2). Each downloads a single small script, installs a private Node 24 without admin rights, then runs the starter, which clones the repository if you are not already inside it.

OS	Command (paste into a terminal)
Windows (PowerShell)	`irm https://raw.githubusercontent.com/open-mercato/open-mercato/main/packages/starter/platform/start.ps1 | iex`
macOS / Linux (bash)	`curl -fsSL https://raw.githubusercontent.com/open-mercato/open-mercato/main/packages/starter/platform/start.sh | bash`

The Windows bootstrap uses PowerShell's built-in downloader (schannel), so a corporate certificate deployed by IT is trusted automatically. The macOS/Linux bootstrap uses the system `curl` for the same

reason.

A. Completely clean machine (no Node, no Git, no repository)

1. Make sure a container runtime is installed and running (Docker Desktop or Rancher Desktop — see section 2; `doctor` will tell you if it is missing).
2. Open **PowerShell** and paste the Windows one-liner above. It installs a private Node 24, fetches the starter, and clones the repository into `%USERPROFILE%\open-mercato`.
3. When asked, pick your LLM provider and paste its API key (input is hidden). This powers the AI assistant.
4. Wait. The **first run takes several minutes** while dependencies install, packages build, the OpenCode image is pulled, and the database is seeded. Every later start takes seconds.
5. At the end the console prints all URLs and the **superadmin login and password**. Open `http://127.0.0.1:3000/backend` and sign in.

B. You already have the repository (git clone or ZIP)

1. If you already have Node 24 + Yarn: run `yarn om` in the repository root.
2. If you do not have Node: double-click `packages\starter\platform\start.cmd`, or run the one-liner from the table above — either installs Node first.
3. Follow the LLM prompt and wait for the summary, as in option A.

> `yarn om`, `npx @open-mercato/starter`, `start.cmd`, and the `irm … | iex` one-liner all run the **same** starter — they differ only in how they guarantee Node first. Inside a clone they all use that clone's vendored starter, so the tool always matches the code it drives.

4. What happens, step by step

The default command (`up`) runs an idempotent converge pipeline. Completed steps are skipped on every re-run, so it is always safe to run again after fixing something.

Step	What it does	Expected duration
Toolchain	Verifies Node 24 + Git; activates the pinned Yarn via corepack	instant
Container runtime gate	Confirms Docker/Rancher is running; if not, prints the exact fix and stops (it never installs the runtime or WSL2)	instant
Corporate TLS trust	Probes real egress hosts; if interception is detected, captures the corporate CA and provisions it into Node, Yarn, Git, and image builds	a few seconds
`.env` / secrets	Generates `.env` secrets once and never overwrites them; prompts for your LLM provider	instant + your input
Install & build	`yarn install`, then builds workspace packages and generates artifacts	first run several min

Infra containers	Pulls the OpenCode base image, then starts OpenCode + PostgreSQL + Redis + Meilisearch (waits for health)	first run several min
Database	Applies migrations and seeds initial data	1–2 min
Dev runtime	Starts `yarn dev` — the app, queue workers, scheduler, and MCP server on your machine	seconds

During long waits the console shows a spinner with elapsed time and a plain-language status. Build progress is also live at `http://127.0.0.1:4000` (blank until the dependency install finishes — that is expected).

> **Memory (WSL2):** both Docker Desktop and Rancher Desktop run every container inside one WSL2 virtual machine, and Windows sizes that VM poorly for this stack. If the first build struggles on a low-memory machine, create or edit `C:\Users\<you>\.wslconfig` with a sensible cap and restart the engine:
>
> ```
> [wsl2]
> memory=10GB
> swap=8GB
> autoMemoryReclaim=gradual
> ```
>
> Then quit the runtime and run `wsl --shutdown` to apply. (Unlike earlier launchers, the starter does not edit `.wslconfig` for you — it never changes machine-level settings.)

5. After setup — URLs and daily commands

Service	URL
Admin app	`http://127.0.0.1:3000/backend`
Build/status splash	`http://127.0.0.1:4000`
MCP server health	`http://127.0.0.1:3001/health`
OpenCode agent health	`http://127.0.0.1:4096/global/health`
Keycloak (dev SSO, opt-in)	`http://127.0.0.1:8080` — start with `yarn om infra up --profile sso`

All commands below work as `yarn om <command>` inside a clone, or `npx @open-mercato/starter <command>` from anywhere.

Action	Command
Start (or resume) the environment	`yarn om` (shorthand for `yarn om up`)
Start in the background (keeps running after you close the terminal)	`yarn om up --detach`
Everything in containers instead of hybrid	`yarn om up --mode docker`

Stop the app + services (data preserved)	`Ctrl` + `C` in the foreground, or `yarn om stop`
Stop the app but leave the containers up	`yarn om stop --keep-infra`
Status of processes, containers, health, URLs	`yarn om status`
Follow the dev logs	`yarn om logs --follow`
Read-only machine audit (+ IT hand-off sheet)	`yarn om doctor`
Just the infra containers	`yarn om infra up` / `yarn om infra down`
Rebuild the OpenCode image (e.g. after `yarn generate`)	`yarn om infra up --rebuild`
Full reset — **DELETES all data** (asks first)	`yarn om reset`

Useful flags on `up`: `--non-interactive`, `--skip-llm-prompt`, `--skip-db`, `--no-infra` (run the app against services you point `.env` at), `--profile <name>` (e.g. `sso`, `storage-s3`), and `-- <args>` to pass extra arguments through to `yarn dev`.

6. Working in your IDE

In the default **hybrid** mode the app runs natively on Windows, so `node_modules` lives in your project folder on disk. **TypeScript autocomplete, go-to-definition, and ESLint work out of the box** in any Windows IDE — VS Code, JetBrains, Cursor — with no extra setup. Saving a file hot-reloads the app immediately. WSL never enters your workflow: do not run the project from a WSL terminal and do not keep a second copy inside a WSL distro.

Tip: "Go to definition" on `@open-mercato/*` imports lands in the monorepo package *sources* — that is expected and usually what you want.

> **In `--mode docker`** the app runs inside a container, so `node_modules` is in a container volume rather than your folder. For full IntelliSense, either install the VS Code "Dev Containers" extension and *Attach to Running Container* → `app` → open `/app`, or run `corepack enable && yarn install` once in the folder for the editor's sake. This is not needed in hybrid mode.

7. Troubleshooting

> **First move:** run `yarn om doctor`. It is read-only and pinpoints most issues (runtime, WSL2, proxy/TLS, ports, container→host connectivity) with an exact remediation for each. The dev logs are at `.mercato/logs/`; tail them with `yarn om logs --follow`. Container logs: `docker compose --project-directory . -f starters/docker/compose.infra.yml logs -f opencode` (swap `opencode` for `postgres`, `redis`, `meilisearch`).

Prerequisites and runtime

Symptom	Cause & fix
“Container runtime — not usable” / setup stops at the runtime gate	Docker/Rancher is not installed or not running. The starter never installs it. On machines where Docker Desktop licensing is not permitted, use **Rancher Desktop** (per-user, free; pick the *dockerd (moby)* engine, Kubernetes off). Start the runtime, then re-run — completed steps are skipped.
“WSL2 is not functional”	WSL2 is required by both runtimes and needs admin + a reboot. Hand the IT sheet from `doctor` to your administrator (`wsl --install --no-distribution` , enable Virtual Machine Platform + WSL, plus CPU virtualization in firmware), reboot, then re-run.
“git not found”	Install Git for Windows from `https://git-scm.com/download/win` (user-scope install works without admin), then re-run.
`yarn install` fails building native modules (better-sqlite3, isolated-vm, node-gyp errors)	Hybrid mode compiles native addons on the host — install the Visual C++ toolchain (admin), then open a new terminal and re-run: `winget install Microsoft.VisualStudio.2022.BuildTools --override "--wait --quiet --add Microsoft.VisualStudio.Workload.VCTools --includeRecommended"` . No admin? Use `yarn om up --mode docker` — native modules build inside the container instead. `yarn om doctor` checks this and prints the exact command.
“PowerShell is running in ConstrainedLanguage mode”	An AppLocker/WDAC policy restricts PowerShell. Ask IT to allow the script or run it from an exempted path.
Node install fails behind a proxy	Set `HTTPS_PROXY` for the session, or point `OM_NODE_DIST_MIRROR` at an internal mirror of `nodejs.org/dist` , then re-run.

Corporate network (proxy / TLS interception)

Symptom	Cause & fix
Doctor reports “Corporate TLS interception”	Normal on managed networks. The starter captures the interception CA automatically and provisions it into Node, Yarn, Git, and image builds. If your organization has an official CA bundle, hand it to the starter via `starters/company/config.mjs` for the most reliable result.
Image pulls fail with certificate errors, but tooling works	The container engine also needs the corporate CA. Docker Desktop reads the Windows certificate store (restart it after the CA is deployed; version **4.42.0+** is required for Zscaler's certificates). Rancher Desktop needs the CA in its WSL distro — the starter writes a provisioning script for it.
Network egress blocked	Ask IT to allow developer egress to `registry.yarnpkg.com` , `github.com` , and `registry-1.docker.io` , or provide internal mirrors (configurable under `starters/company/`).

AI assistant and connectivity

Symptom	Cause & fix
AI chat (Ctrl+K) shows authorization errors / MCP "disconnected"	The OpenCode container reads the MCP key only at start; the key rotates when the database is reset. Restart it: `docker compose --project-directory . -f starters/docker/compose.infra.yml restart opencode`. Verify with `curl http://127.0.0.1:4096/mcp` → should show `"status":"connected"`.
Doctor: "the compose extra_hosts pin does NOT reach the host" (common on Rancher Desktop / WSL2)	The OpenCode container reaches the host MCP server via `host.docker.internal`, which on some WSL2 setups resolves to the WSL distro instead of Windows. Doctor prints the fix: set `OM_HOST_GATEWAY=<ip>` in the repo-root `.env`, then restart the OpenCode container.
Windows Firewall prompt on first start	The MCP server listens on the host; Windows asks once whether to allow `node.exe`. Allow it on **Private networks** so containers can reach it.
AI chat gives provider/model errors	Check your provider key in the repo-root `.env` (e.g. `OPENAI_API_KEY`) and `OM_AI_PROVIDER`, then restart OpenCode (command above).

Application, data, and ports

Symptom	Cause & fix
First boot "looks stuck"	The first run installs dependencies, builds packages, pulls the OpenCode image, and seeds the database — several minutes. Watch `http://127.0.0.1:4000` or `yarn om logs --follow`. It fails loudly if something actually breaks.
"Port 3000/3001/4096… already in use"	Another process owns the port (a local PostgreSQL on 5432 is common). Stop leftovers with `yarn om stop`, or override the port in the repo-root `.env` (`APP_PORT`, `MCP_PORT`, `OPENCODE_PORT`, `POSTGRES_PORT`, `OM_DEV_SPLASH_PORT`) and re-run.
Login credentials from the summary do not work	If you recreated `.env` while keeping the old database volume, the printed password may differ from the seeded one. Cleanest fix: `yarn om reset` (deletes data), then a fresh run.
Containers keep dying on a 16 GB machine	Usually the WSL2 VM running out of memory — set a `.wslconfig` cap (see the note in section 4), close other heavy apps during the first build, then `wsl --shutdown` and re-run.
New agents/skills not visible after `yarn generate`	OpenCode loads them on new sessions from a bind mount. Restart it: `docker compose --project-directory . -f starters/docker/compose.infra.yml restart opencode`.

8. FAQ

Do I need to install Node.js first?

No. The one-line bootstrap (`irm … start.ps1 | iex` on Windows) installs a private, checksum-verified Node 24 without administrator rights, then runs the starter. Only `yarn om` / `npx @open-mercato/starter` assume Node already exists.

Do I need administrator rights?

Not for the starter itself. Admin is only needed once, by you or IT, to enable WSL2 and install the container runtime — the starter never does this and instead prints exactly what IT must approve (`yarn om doctor`).

We are not allowed to use Docker Desktop (licensing). What now?

Use **Rancher Desktop** (per-user, free). Install it with the *dockerd (moby)* engine and Kubernetes off, start it, then run the starter normally.

Hybrid vs. docker mode — which should I use?

Use the default **hybrid** mode for everyday development: it is faster and your IDE gets full IntelliSense. Use `up --mode docker` only when policy forbids running Node workloads directly on the host.

Where are my login credentials?

In the summary at the end of setup, and in the repo-root `.env` (`OM_INIT_SUPERADMIN_EMAIL` / `OM_INIT_SUPERADMIN_PASSWORD`). They are not written to any log file.

Where does my data live? How do I wipe it?

In named Docker volumes (they survive stops and reboots). Wipe everything with `yarn om reset` (asks for confirmation).

How do I update to the latest code?

`git pull` , then `yarn om` . The starter reconverges — installs new dependencies, applies new migrations, and restarts.

Can I change the AI provider later?

Yes — edit the repo-root `.env` (provider key + `OM_AI_PROVIDER` , optionally `OM_AI_MODEL`), then restart OpenCode: `docker compose --project-directory . -f starters/docker/compose.infra.yml restart opencode` .

Which ports are used?

3000 (app), 4000 (build splash), 3001 (MCP), 4096 (OpenCode), 5432 (PostgreSQL), 6379 (Redis), 7700 (Meilisearch), 8080 (Keycloak, opt-in). All overridable in `.env` .

Does it work offline?

The first run needs the internet for Docker images and npm packages. After a successful first boot, day-to-day work is local.